

Validation checks for the GLM path

The tool contract currently specifies inputs, parameters and outputs. Assumptions, failure modes and validation tests are named in the spec but not yet defined. These are proposed contents for those three sections: at each pipeline stage, the checks that fire automatically and the plots a person should look at.

PROPOSALS

Define assumptions, failure modes and validation tests for the GLM tool, as a worked example for the other tools.

Each check carries a threshold and a consequence, so they are enforceable rather than advisory. Most of the underlying numbers already exist in the tool's outputs.

Emit one self-contained HTML report per run, carrying every diagnostic below.

Two PNGs is not enough to judge a fit. A single report, stored as a run artifact next to `fit.npz`, gives the researcher one thing to open and gives the approval step something to attach. Checks that fired appear inline beside the plot that shows why.

Implement the conversion from sorted units to a binned table.

The schema marks `sorted_units.nwb` as not consumed directly, with the conversion described in prose. That step constructs the design matrix: bin width, predictor construction and trial alignment are all decided there.

Declare what a response means across sessions — matched, or session-scoped.

Unit 1 in one session is not unit 1 in the next unless units were explicitly tracked. The tool fits one model per response *column* and has no concept of a unit, so the identity assumption lives in how the table was written rather than anywhere in the code. Pooling is sound for photometry, where a channel really is the same sensor each session; it is not sound for sorted units.

Declare whether co-recorded responses may enter the design as predictors.

A task-only model asks what an event explains about a neuron. A coupled model, with neighbouring units in the design, asks what the event explains *beyond what those neighbours already explain* — shared task drive is absorbed by the coupling terms, so task kernels shrink and can vanish. Same ΔD^2 , different claim. Predictors and responses are both just column lists over one table, so nothing currently distinguishes the two models, and nothing prevents a response from predicting itself.

Declare which predictors are dropped together to form the subset models.

ΔD^2 is the cost of removing a predictor *given everything else in the model*, so the leave-out grouping is the hypothesis, not a formatting detail. Correlated predictors dropped one at a time each look negligible, because the others absorb them; dropped as a block they can be decisive — the per-predictor default systematically understates any correlated set. Current scope does not expose this: `jaxGLM` hardcodes one group per predictor (`run_pipeline.py` splits column names on the predictor stem), and Max's schema exposes `predictor_importance` only as an on/off boolean. No parameter, on either side, groups predictors into blocks, compares nested models, or drops part of a kernel.

CHECKS AND PLOTS, IN PIPELINE ORDER

GATE stop the run **WARN** flag loudly **LOOK** human judgement

N = events, L = lags per predictor, U = units, P = predictor columns.

Census

GATE

Event count per predictor, overall and per session.

fail: N = 0 in any session being pooled

Predictor variance.

fail: zero variance (constant column)

Missing values in any predictor or response column.

fail: NaN fraction > 0 with no stated imputation policy

Per-session presence, where sessions are pooled without a session term.

warn: predictor present in < 100% of sessions

Response viability.

fail a unit: mean rate < 0.5 Hz, or < 100 spikes total

LOOK AT

Event count per predictor, split by session. Predictors that vanish in some sessions, or are too rare to support a kernel.

Firing-rate distribution across units. A pile-up at zero means dead or barely-active units are entering the fit.

Pooling semantics

GATE

Declare what a response column means across sessions: `shared` (the same physical signal every session — a photometry channel, a tracked unit) or `session_scoped` (identity holds only within a session — the default for sorted units).

fail: undeclared when more than one session is pooled

Pooling is valid only for `shared` . Session-scoped responses must be fitted one session at a time, with

kernels aggregated afterwards — there is no correct way to pool them in a single call.

fail: session_scoped with more than one session in the table

Claiming shared for sorted units asserts units were tracked across sessions, which the dataset metadata must record.

fail: shared declared for sorted units with no cross-session matching recorded

Neither path through the current code is safe, and the safer one only stops by accident.

reused names (incl. via responses_prefix) → clean concat → one fit over unrelated neurons

session-namespaced names → NaN gaps → check_design stops the run

LOOK AT

Response column against session, as a presence grid. Reused names show as a filled block, session-scoped names as a diagonal. It is immediately obvious which regime the table is in.

Design matrix

GATE

Events per free parameter. Each event constrains the whole kernel at once, so N events give roughly N independent constraints on L lag weights.

fail: $N < L$ · warn: $N < 10L$

Instantaneous correlation between predictor *groups* — not between lags within a group, which are correlated by construction.

fail: $|r| > 0.95$ · warn: $|r| > 0.8$ or $VIF > 10$

Temporal locking — the fixed-lag confound the row above misses. Two events that never share a bin but sit at a near-constant offset have zero-lag $|r| \approx 0$, yet after the shift expansion A_{lag+k} and $B_{lag+(k-L)}$ are the same column, so their kernels cannot be separated. Test the nearest-neighbour interval distribution per ordered pair.

warn: a partner event at a near-constant offset (interval IQR < ~2 bins) for > 80% of occurrences
the separability lever is interval variability — if the task never varies the A→B gap, no amount of data separates the kernels; the exactly-fixed case falls through to the rank / conditioning gate below

Rank and conditioning of X. Elastic net returns an answer regardless, so this asks whether the data determine the weights or the penalty does.

fail: rank < P, or condition number > 1e10

Lag windows against session and trial boundaries.

fail: any window crosses a boundary unmasked

A response must never appear in its own predictor list. With the default lag window spanning zero this is exact self-prediction and D^2 goes to 1.

fail: any response column also listed in data.predictors for that fit

unguarded today — check_design never receives response names, and config validation only tests that the lists are non-empty

Declare model scope: task-only, or coupled with co-recorded units in the design. Coupling terms must be strictly causal, or a neighbour's simultaneous activity predicts the target.

fail: undeclared when non-task responses appear as predictors · fail: coupling term at lag ≥ 0

Coupled predictors sharing an electrode with the response — spike-sorting contamination produces

coupling that looks like connectivity.

warn: coupling predictor from the same channel as the response

LOOK AT

An image of a slice of X. Construction bugs are visible here and almost nowhere else — all-zero columns, misaligned shifts, blocks that should not be there.

Correlation heatmap between predictor groups. Bright off-diagonal blocks mean you cannot attribute an effect to one event rather than another.

Pairwise event cross-correlograms. A sharp peak at any lag — not only zero — means two events are temporally locked and their kernels are confounded; the peak's width is how badly.

If coupled: the unit-by-unit coupling weight matrix. Strong weights concentrated on same-electrode pairs are sorting contamination, not connectivity.

Alignment

WARN

Raw peri-event firing-rate average per predictor, computed without the model, scored against a pre-event baseline.

warn: kernel significant but raw peak $|z| < 2$

Streams on different clocks — regress each stream's shared event times on the other.

fail: $\max |\Delta t| > 1 \text{ bin}$, or $|\text{slope} - 1| > 1e-4$

LOOK AT

Raw peri-event average per predictor, no model involved. If nothing is visible here, a significant kernel is telling you about the model, not the neuron.

Event times of one stream against the other, with the identity line. A slope that is not 1 is clock drift, and no other diagnostic will show it.

Fit diagnostics

WARN

Solver convergence across units.

fail: $< 80\%$ converged · warn: $< 95\%$

Selected alpha against the ends of the search grid — at an edge, the grid is wrong and the answer is not the one CV would have chosen.

warn: $> 5\%$ of units select the first or last grid point

Shape of the CV deviance curve.

warn: minimum within 1 SE of both grid ends (no selection happening)

Held-out D^2 across the population.

warn: $> 10\%$ of units have held-out $D^2 < 0$

Report in-sample and held-out D^2 separately and label them. Both are computed today; neither file says which is which.

fit.npz d2 and the summary.csv D2 column are IN-SAMPLE

held-out d2_full comes from predictor_importance; stdout says "in-sample", the files do not

LOOK AT

LOOK AT

In-sample against held-out D^2 , one point per unit, with the identity line. The single most informative plot here. Points hugging the line generalise; points falling well below it are overfitting; points below zero do worse than predicting the mean.

CV deviance against alpha, with the selected value and both grid ends marked. You want a real minimum sitting away from the edges.

Regularisation balance

WARN

Reconstruction against penalty on the same scale. `objective_terms` returns `penalty_frac`; use the deviance-based `recon`, not `nll`.

warn: `penalty_frac < 0.01` (decorative) or `> 0.5` (penalty dominates)

Read alongside alpha relative to `alpha_max`, because the penalty term is non-monotonic — it vanishes both as $\alpha \rightarrow 0$ and at `alpha_max` where every weight is zero, so the fraction alone cannot separate under- from over-regularisation.

warn: `alpha/alpha_max < 1e-3` or `> 0.5`

Fixed alpha combined with a changed `standardize` setting. Standardising X makes a raw-scale alpha roughly 100x too weak.

fail: alpha is a fixed float and `standardize` differs from the run alpha was chosen under

LOOK AT

Reconstruction against penalty, one point per unit, log axes, with the $y = x$ line. The two terms in the same units, which is what “same scale?” actually asks. Points far below the line mean the penalty is decorative; far above, it is deciding the answer. Colour by `alpha/alpha_max` to break the non-monotonicity — a point near the origin is either barely regularised or fully crushed, and the colour says which.

Fraction of weights driven exactly to zero. None means the sparsity penalty is not biting; nearly all means it has eaten the model.

Is it real

LOOK

Effect size, not significance alone.

label weak: $\Delta D^2 / D^2 < 0.02$, whatever the p-value

ΔD^2 is conditional on the leave-out grouping (see proposals) — report the grouping beside the number, or it cannot be compared across runs.

LOOK AT

Model-predicted peri-event average overlaid on the raw one. The plot that settles arguments. If the model claims an event drives the neuron, the raw data has to show it too.

Kernels with error bands, per predictor. A kernel whose band spans zero throughout is not a result.

Predicted against actual firing rate over a stretch of time. Shows whether the model tracks the data or just its mean.

ΔD^2 per predictor across units. The distribution of effect sizes, which is what a reader needs and a p-value never gives.

Controls

GATE

Circular-shift null per predictor.

fail: observed ΔD^2 below the 95th percentile of the shifted null

Trial-time-only baseline — a model with position-in-trial and no events at all.

fail: event model does not beat it by $\Delta D^2 > 0.01$ and significantly